

Lecture 1

Programming Paradigms, Types of Programming Paradigms, POP vs OOP

Subject: Object-Oriented Programming with Java (DI03016051)

Learning Outcomes

After this lecture, students will be able to:

- Define programming and programming paradigm.
 - Explain different programming paradigms.
 - Differentiate between POP and OOP.
 - Identify the advantages and limitations of POP and OOP.
-

1. What is Programming?

Programming is the process of writing instructions that tell a computer what to do. A computer cannot think on its own. It only follows the instructions written by a programmer.

Example

- ATM Software
- Calculator
- WhatsApp
- Railway Reservation System

All these are programs created using programming languages.

GTU Definition

Programming is the process of writing, testing and maintaining instructions that enable a computer to perform a specific task.

2. What is a Programming Paradigm?

A **Programming Paradigm** is a style or method of writing computer programs. Different programmers can solve the same problem using different approaches.

Real-life Example

Suppose you want to travel from Ahmedabad to Rajkot.

You can travel by:

- Bus
- Train
- Car

The destination is the same, but the method is different.

Similarly, software can be developed using different programming styles called

Programming Paradigms.

3. Types of Programming Paradigms

The commonly used programming paradigms are:

1. Procedure-Oriented Programming (POP)

The program is divided into **functions**.

Example Languages

- C
 - Pascal
 - FORTRAN
-

2. Object-Oriented Programming (OOP)

The program is divided into **objects**.

Example Languages

- Java
 - C++
 - C#
 - Python
-

4. Procedure-Oriented Programming (POP)

In POP, the program is divided into small functions. Each function performs a specific task.

Example

A College Management System may have functions such as:

- Add Student
- Search Student
- Calculate Result
- Print Report

Each task is handled by a separate function.

Characteristics of POP

- Program is divided into functions.
- Follows **Top-Down** approach.
- Data is shared among functions.
- Suitable for small programs.

Advantages

- Easy to learn.
- Fast for small applications.
- Less memory required.

Limitations

- Poor data security.
 - Difficult to maintain large programs.
 - Code reuse is limited.
-

5. Object-Oriented Programming (OOP)

In OOP, the program is divided into **objects**.

Each object contains:

- **Data (Attributes)**
- **Methods (Functions)**

Example

Car Object

Data

- Brand
- Color
- Speed

Methods

- Start()
- Stop()
- Brake()

This closely resembles real-world objects.

Characteristics of OOP

- Program is divided into objects.
- Data and methods are kept together.
- Supports code reuse.
- Provides better security.

6. POP vs OOP

POP	OOP
Program divided into functions	Program divided into objects
Top-Down approach	Bottom-Up approach
Data is less secure	Data is more secure
Difficult to reuse code	Easy to reuse code
Suitable for small programs	Suitable for large programs
Example: C	Example: Java

Exam Tip: This comparison is one of the most frequently asked GTU questions.

Key Points to Remember

- **Programming** means writing instructions for a computer.
- A **Programming Paradigm** is a style of programming.
- POP focuses on **functions**.
- OOP focuses on **objects**.
- Java follows the **Object-Oriented Programming** paradigm.

Summary

- A computer works according to programs written by programmers.
- Programming paradigms provide different approaches to software development.
- POP is suitable for small applications, while OOP is preferred for large and complex software.
- Java is an Object-Oriented Programming language.

GTU Questions**Short Questions**

1. Define Programming.
2. What is a Programming Paradigm?
3. Define Procedure-Oriented Programming.
4. Define Object-Oriented Programming.

Long Questions

1. Explain different programming paradigms.
 2. Compare POP and OOP.
-

Lecture – 2

Features of Object-Oriented Programming (OOP), Benefits and Applications of OOP

Subject: Object-Oriented Programming with Java (DI03016051)

Unit: 1 – Introduction to Object-Oriented Programming Paradigm and Java

Learning Outcomes

After completing this lecture, students will be able to:

- Define a **Class** and an **Object**.
 - Explain the four main features (pillars) of OOP.
 - Describe the benefits of Object-Oriented Programming.
 - List the applications of OOP in real-world software.
-

Introduction

In the previous lecture, we learned that **Object-Oriented Programming (OOP)** is a programming paradigm that organizes a program into **objects** instead of functions.

To understand OOP completely, we first need to understand two important terms:

- **Class**
- **Object**

These are the building blocks of every Java program.

1. Class

A **Class** is a **blueprint** or **template** used to create objects. It defines the **properties (data)** and **behaviour (methods)** of an object.

Simple Example

Before constructing a house, an engineer prepares a **house plan (blueprint)**.

Many houses can be built using the same blueprint.

Similarly,

- **Blueprint → Class**
- **House → Object**

Student Class Example

Student

Properties

Roll Number

Name

Branch

Methods

Study()
AttendClass()
DisplayResult()

The class only defines what a student object should contain. No actual student exists until an object is created.

GTU Definition

A class is a user-defined blueprint or template that defines the properties (data) and methods (functions) of objects.

2. Object

An **Object** is an **instance of a class**. It represents a real-world entity and contains actual values for the properties defined in the class.

Example

If **Student** is a class, then:

- Rahul (Roll No. 101)
- Priya (Roll No. 102)

are **objects** of the Student class.

Real-Life Example

Think about a **car showroom**.

There is one **car design (class)**, but many actual cars (objects) are manufactured using that design.

GTU Definition

An object is an instance of a class that contains actual data and can perform actions defined by the class.

Difference Between Class and Object

Class	Object
Blueprint or template	Actual instance
Logical entity	Physical entity
No memory allocated	Memory allocated
Used to create objects	Created from a class

3. Features of Object-Oriented Programming

The main features of OOP are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

These are known as the **Four Pillars of OOP**.

3.1 Encapsulation

Encapsulation means **combining data and methods into a single unit (class)** and protecting the data from direct access.

Real-Life Example

An **ATM Machine** hides all the internal processes like balance verification and transaction processing. The user only interacts through buttons and the display. Similarly, in Java, data is protected inside a class and accessed using methods.

Advantages

- Protects data.
- Improves security.
- Prevents unauthorized access.
- Makes programs easier to maintain.

GTU Definition

Encapsulation is the process of wrapping data and methods together into a single unit called a class.

3.2 Abstraction

Abstraction means **showing only the essential information and hiding unnecessary implementation details.**

Real-Life Example

When driving a **car**, the driver uses the steering wheel, accelerator, and brake. The driver does not need to know how the engine works internally.

Similarly, Java hides unnecessary implementation details and provides only the required functionality to the user.

Advantages

- Reduces complexity.
- Makes programs easier to understand.
- Protects internal implementation.

GTU Definition

Abstraction is the process of hiding implementation details and showing only the necessary features to the user.

3.3 Inheritance

Inheritance allows one class to acquire the properties and methods of another class. It promotes **code reusability**.

Real-Life Example

A child inherits characteristics such as eye color, height, or surname from parents.

Similarly, in Java, a new class can inherit properties and methods from an existing class.

Example

Vehicle

↓

Car

↓

SportsCar

The **SportsCar** class inherits the features of **Car**, and **Car** inherits the features of **Vehicle**.

Advantages

- Reuses existing code.
- Reduces code duplication.
- Simplifies maintenance.

GTU Definition

Inheritance is the process by which one class acquires the properties and methods of another class.

3.4 Polymorphism

The word **Polymorphism** comes from two Greek words:

- **Poly** = Many
- **Morph** = Forms

It means **one entity can take many forms**.

Real-Life Example

A teacher may have different roles:

- Teacher in the classroom
- Parent at home
- Customer in a shop

The same person performs different roles depending on the situation.

Similarly, in Java, the same method name can perform different tasks depending on how it is used.

Advantages

- Increases flexibility.
- Improves code readability.
- Makes programs easier to extend.

GTU Definition

Polymorphism is the ability of an object or method to take many forms and perform different tasks.

4. Benefits of Object-Oriented Programming

OOP offers several advantages over Procedure-Oriented Programming.

1. Code Reusability

Existing classes can be reused using inheritance, reducing duplicate code.

2. Data Security

Encapsulation protects important data from unauthorized access.

3. Easy Maintenance

Changes made in one class have minimal impact on other classes.

4. Modularity

Large programs are divided into smaller classes, making them easier to manage.

5. Scalability

New features can be added without rewriting the entire program.

6. Better Real-World Representation

Objects in software can represent real-world entities like **Student**, **Employee**, **Car**, or **Bank Account**.

5. Applications of OOP

OOP is widely used in modern software development.

Some common applications include:

- Banking Systems
 - Hospital Management Systems
 - Library Management Systems
 - School and College Management Systems
 - E-commerce Applications (Amazon, Flipkart)
 - Android Mobile Applications
 - Desktop Applications
 - Game Development
-

OOP Features at a Glance

Feature	Purpose	Real-Life Example
Class	Blueprint for objects	House blueprint
Object	Actual entity	Constructed house
Encapsulation	Combine data and methods	ATM machine
Abstraction	Hide internal details	Driving a car
Inheritance	Reuse existing features	Child inheriting traits
Polymorphism	One entity, many forms	Teacher with different roles

Key Points to Remember

- A **Class** is a blueprint for creating objects.
 - An **Object** is an instance of a class.
 - **Encapsulation** protects data by combining it with methods.
 - **Abstraction** hides unnecessary implementation details.
 - **Inheritance** promotes code reuse.
 - **Polymorphism** allows one interface or method to perform different tasks.
 - These four concepts form the foundation of Object-Oriented Programming.
-

Summary

- OOP is based on **classes** and **objects**.
 - A class defines the structure, while an object represents a real-world entity.
 - The four pillars of OOP are **Encapsulation, Abstraction, Inheritance**, and **Polymorphism**.
 - OOP improves code reuse, security, maintainability, and scalability.
 - OOP is widely used in modern software such as banking systems, Android applications, and e-commerce platforms.
-

GTU Examination Questions

Short Questions

1. Define a class.
2. Define an object.
3. What is encapsulation?
4. What is abstraction?
5. Define inheritance.
6. What is polymorphism?

Long Questions

1. Explain the four pillars of Object-Oriented Programming with suitable examples.
 2. Explain the benefits and applications of Object-Oriented Programming.
 3. Differentiate between a class and an object.
-

Lecture – 3

Introduction to Java: History, Evolution, Features, Editions and Applications

Subject: Object-Oriented Programming with Java (DI03016051)

Unit: 1 – Introduction to Object-Oriented Programming Paradigm and Java

Learning Outcomes

After completing this lecture, students will be able to:

- Explain the background and history of Java.
 - Describe the evolution of Java.
 - List and explain the major features of Java.
 - Identify the different editions of Java.
 - Explain the major applications of Java.
-

1. Introduction to Java

Java is a high-level, general-purpose programming language developed to create software applications for different types of devices and platforms.

Java is widely used for:

- Web applications
- Mobile applications
- Desktop software
- Enterprise applications
- Banking systems
- Games
- Cloud applications

Java is also an **Object-Oriented Programming language**, which means it uses concepts such as:

- Classes
 - Objects
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Abstraction
-

2. Background and History of Java

Java was developed at **Sun Microsystems** in the early 1990s.

The project was initially led by **James Gosling**, along with a team of developers.

Important Timeline

Year	Event
1991	Java project started

Year	Event
------	-------

1991	Initially named Oak
------	----------------------------

1995	Officially renamed Java
------	--------------------------------

1995	Java 1.0 released
------	-------------------

2010	Oracle acquired Sun Microsystems
------	----------------------------------

Present Java continues to be developed under Oracle and the Java community

Why Was Java Developed?

Initially, Java was designed for electronic devices and embedded systems.

The developers wanted a language that could:

- Work on different types of devices.
- Be portable.
- Be reliable.
- Support network-based applications.

Later, Java became widely used for:

- Web development
- Enterprise software
- Mobile applications
- Desktop applications

3. Evolution of Java

Java has continuously evolved with new versions and features.

Basic Evolution

Oak

↓

Java

↓

Java 1.0

↓

Java 2

↓

Java 5

↓

Java 8

↓

Modern Java Versions

Important Developments

- **Java 1.0:** Initial public release.
- **Java 2:** Introduced different editions of Java.
- **Java 5:** Introduced features such as Generics and Enhanced for Loop.

- **Java 8:** Introduced Lambda Expressions and Stream API.
- **Modern Java:** Continues to provide regular updates and new features.

✦ **Note:** The syllabus mainly focuses on the basic concepts of Java and its programming environment.

4. Important Features of Java

Java is popular because of its important features.

The commonly discussed features are:

1. Simple
 2. Object-Oriented
 3. Platform Independent
 4. Portable
 5. Secure
 6. Robust
 7. Architecture Neutral
 8. Interpreted
 9. High Performance
 10. Multithreaded
 11. Distributed
 12. Dynamic
-

4.1 Simple

Java is easier to learn compared to languages such as C++ because it removes many complex features.

For example, Java does not support:

- Pointer arithmetic
 - Multiple inheritance through classes
 - Manual memory deallocation
-

4.2 Object-Oriented

Java is based on OOP concepts such as:

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

This makes Java suitable for developing large applications.

4.3 Platform Independent

This is one of the most important features of Java.

A Java program can run on different operating systems without changing the original source code.

Example

Java Program

↓

Bytecode

↓

Windows | Linux | macOS

This is possible because of the **JVM**.

 **Remember:**

Write Once, Run Anywhere (WORA)

4.4 Portable

Java programs can be moved from one system to another easily.

The same Java bytecode can run on different platforms if the required JVM is available.

4.5 Secure

Java provides several security features:

- No direct pointer manipulation.
- Bytecode verification.
- Runtime security checks.
- Controlled access to system resources.

This is important for applications that work over networks and the Internet.

4.6 Robust

Java is considered robust because it provides:

- Strong type checking.
- Exception handling.
- Automatic memory management.
- Garbage collection.

These features help reduce programming errors.

4.7 Architecture Neutral

Java bytecode is not designed for a specific processor or hardware architecture.

Therefore, the same bytecode can run on different hardware systems using the appropriate JVM.

4.8 Interpreted and Compiled

Java source code is first compiled into **bytecode**.

The bytecode is then executed by the JVM.

Source Code

↓

Compiler

↓

Bytecode

↓

JVM

↓

Machine Code

Therefore, Java uses both compilation and interpretation/JIT execution.

4.9 High Performance

Java provides good performance through:

- Just-In-Time (JIT) compilation.
- JVM optimization.
- Efficient memory management.

4.10 Multithreaded

Java supports **multithreading**.

Multithreading means performing multiple tasks at the same time.

Example

A mobile application may:

- Download a file.
- Play music.
- Respond to user input.

These tasks can be performed concurrently.

4.11 Distributed

Java supports the development of applications that communicate over networks.

Examples:

- Web applications
- Distributed systems
- Client-server applications

4.12 Dynamic

Java can load classes and libraries when required during program execution.

This makes Java flexible and suitable for changing environments.

5. Editions of Java

Java is available in different editions for different types of applications.

5.1 Java SE – Standard Edition

Java SE is the basic edition of Java.

It is used for:

- Core Java programming
- Desktop applications
- Basic Java concepts

📌 This course mainly focuses on the fundamentals of **Java SE**.

5.2 Java EE – Enterprise Edition

Java EE is used for large-scale enterprise applications.

Examples:

- Banking systems
- E-commerce applications
- Enterprise web applications

Today, Java EE technologies are commonly associated with the **Jakarta EE** ecosystem.

5.3 Java ME – Micro Edition

Java ME is designed for small and resource-constrained devices.

Examples:

- Embedded devices
 - Small electronic devices
-

Java Editions at a Glance

Edition	Main Use
Java SE	Core Java and desktop applications
Java EE / Jakarta EE	Enterprise and web applications
Java ME	Embedded and resource-constrained devices

6. Applications of Java

Java is used in many areas.

1. Web Applications

Java is used to create:

- Online shopping websites.
 - Banking portals.
 - Government websites.
-

2. Enterprise Applications

Large organizations use Java for:

- Banking software.
- Insurance systems.

- Business applications.
-

3. Android Applications

Java has historically been one of the major languages used for Android application development.

4. Desktop Applications

Java can be used to create:

- Billing software.
 - Management systems.
 - Utility applications.
-

5. Banking Applications

Java is widely used in:

- ATM systems.
 - Online banking.
 - Transaction processing systems.
-

6. Big Data and Cloud Applications

Java is used in several large-scale data processing and cloud-based systems.

7. Game Development

Java can be used to develop games and game-related applications.

8. Embedded Systems

Java can be used in certain embedded and resource-constrained environments.

Key Points to Remember

- Java was initially developed at **Sun Microsystems**.
 - **James Gosling** is commonly known as the father of Java.
 - Java was initially called **Oak**.
 - Java was officially released in **1995**.
 - Java follows the **Object-Oriented Programming** paradigm.
 - Java is famous for **platform independence**.
 - **WORA** means **Write Once, Run Anywhere**.
 - The major editions are **Java SE, Java EE/Jakarta EE, and Java ME**.
 - Java is used in web, enterprise, banking, desktop, mobile, cloud, and other applications.
-

Summary

Java is a powerful, object-oriented, general-purpose programming language.

It became popular because of features such as:

- Platform independence
- Portability
- Security
- Robustness
- Object-oriented design
- Multithreading

Java has evolved from its early **Oak** project into a widely used programming platform for developing applications across many industries.

GTU Examination Questions

Short Questions

1. Who developed Java?
2. What was the original name of Java?
3. When was Java officially released?
4. What is meant by platform independence?
5. What is WORA?
6. List the editions of Java.
7. List any five features of Java.

Long Questions

1. Explain the important features of Java.
 2. Explain the history and evolution of Java.
 3. Explain the different editions and applications of Java.
-

Lecture – 4

JDK, JRE, JVM, Bytecode, Java Compilation Process, Environment Setup and Java Program Structure

Subject: Object-Oriented Programming with Java (DI03016051)

Unit: 1 – Introduction to Object-Oriented Programming Paradigm and Java

Learning Outcomes

After completing this lecture, students will be able to:

- Explain JDK, JRE, and JVM.
 - Understand Java bytecode.
 - Explain the Java compilation and execution process.
 - Understand the basic Java environment setup.
 - Identify the basic structure of a Java program.
-

1. Java Platform

To write and run a Java program, we need a Java environment.

The three important components are:

- **JDK** – Java Development Kit
- **JRE** – Java Runtime Environment
- **JVM** – Java Virtual Machine

These components work together to develop and execute Java programs.

2. JVM – Java Virtual Machine

What is JVM?

JVM stands for **Java Virtual Machine**.

It is a software-based machine that executes Java **bytecode**.

Java source code is not directly executed by the operating system. First, it is converted into bytecode, and the JVM executes that bytecode.

Simple Diagram

Java Program

↓

Bytecode

↓

JVM

↓

Operating System

↓

Hardware

Important Point

Different operating systems have different JVM implementations.

Java Bytecode

↓

↓ ↓ ↓

Windows JVM Linux JVM macOS JVM

This is the main reason Java is platform independent.

3. JRE – Java Runtime Environment

What is JRE?

JRE stands for **Java Runtime Environment**.

It provides the environment required to **run Java applications**.

JRE Contains:

JRE

├─ JVM

└─ Java Class Libraries

Important Point

JRE is required to **run** Java programs.

If a user only wants to run a Java application and does not want to develop one, JRE is sufficient.

4. JDK – Java Development Kit

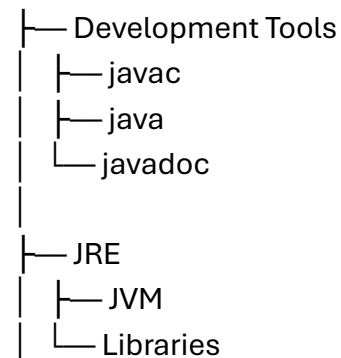
What is JDK?

JDK stands for **Java Development Kit**.

It is required to **develop, compile, debug, and run Java programs**.

JDK Contains:

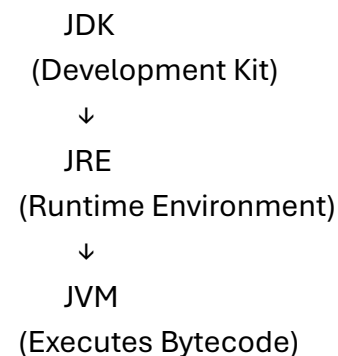
JDK



Important Point

JDK is required by programmers to develop Java applications.

5. Relationship Between JDK, JRE and JVM



Simple Example

Think of preparing food:

- **JDK** → Complete kitchen with all cooking tools.
- **JRE** → Kitchen required to prepare/run the food.
- **JVM** → The actual cooking process that executes the instructions.

6. Difference Between JDK, JRE and JVM

JDK	JRE	JVM
Used to develop Java programs	Used to run Java programs	Executes bytecode

JDK

Contains JRE

Includes development tools

JRE

Contains JVM

Includes libraries

JVM

Part of JRE

Provides execution environment

 **Remember:**

JDK = JRE + Development Tools

JRE = JVM + Java Libraries

7. What is Bytecode?

When a Java program is compiled, it is converted into an intermediate code called **Bytecode**.

Java bytecode is stored in a file with the extension:

.class

Example

Hello.java

↓

Compiler

↓

Hello.class

The .class file contains bytecode.

The JVM reads and executes this bytecode.

8. Java Compilation and Execution Process

Consider the following Java program:

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

Step 1: Write Source Code

The programmer writes:

Hello.java

Step 2: Compile the Program

The Java compiler (javac) compiles the source code.

`javac Hello.java`

The compiler creates:

Hello.class

Step 3: Execute the Program

The JVM runs the bytecode.

```
java Hello
```

Output:

```
Hello Java
```

Complete Process

```
Hello.java
```

↓

```
javac Compiler
```

↓

```
Hello.class
```

(Bytecode)

↓

```
JVM
```

↓

```
Machine Instructions
```

↓

```
Output
```

9. Java Environment Setup

To develop Java programs, we generally need:

1. Java Development Kit (JDK)
2. Text Editor or IDE

Common Java Development Tools

- IntelliJ IDEA
 - Eclipse
 - NetBeans
 - Visual Studio Code
 - Command Prompt / Terminal
-

Basic Setup Steps

Step 1: Install JDK

Install a suitable JDK on the computer.

Step 2: Verify Installation

Open Command Prompt or Terminal and type:

```
java -version
```

This checks whether Java Runtime is available.

To check the compiler:

```
javac -version
```

Step 3: Write a Java Program

Create a file such as:

Hello.java

Step 4: Compile and Run

```
javac Hello.java
```

```
java Hello
```

10. Basic Structure of a Java Program

Example:

```
class Hello {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello Java");  
  
    }  
  
}
```

Let's understand the important parts.

10.1 Class Declaration

```
class Hello
```

This creates a class named Hello.

A Java program generally contains one or more classes.

10.2 main() Method

```
public static void main(String[] args)
```

The main() method is the starting point of a standalone Java application.

Program execution begins from the main() method.

10.3 System.out.println()

```
System.out.println("Hello Java");
```

This statement displays output on the screen.

Output:

Hello Java

10.4 Curly Braces { }

Curly braces define the beginning and end of a block.

```
class Hello {  
    // Class body
```

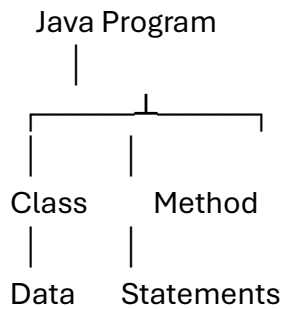
}

10.5 Semicolon ;

Most Java statements end with a semicolon.

`System.out.println("Hello Java");`

11. Basic Java Program Structure



A simple Java program generally contains:

- Class
 - Main method
 - Statements
 - Variables
 - Comments (optional)
-

Key Points to Remember

- **JDK** is used to develop Java programs.
 - **JRE** is used to run Java programs.
 - **JVM** executes Java bytecode.
 - Java source files have the extension `.java`.
 - Compiled Java bytecode files have the extension `.class`.
 - `javac` is used to compile Java programs.
 - `java` is used to run Java programs.
 - Program execution generally begins from the `main()` method.
-

Summary

The Java development and execution process is:

Source Code (`.java`)

↓

`javac`

↓

Bytecode (`.class`)

↓

JVM

↓

Output

The relationship between Java components is:

JDK

↓

JRE

↓

JVM

Java achieves platform independence because the same bytecode can be executed by JVMs designed for different operating systems.

GTU Examination Questions

Short Questions

1. What is JVM?
2. What is JRE?
3. What is JDK?
4. What is bytecode?
5. What is the extension of a Java source file?
6. What is the extension of a Java bytecode file?
7. What is the purpose of javac?
8. What is the purpose of the main() method?

Long Questions

1. Explain JDK, JRE, and JVM with a suitable diagram.
2. Explain the Java compilation and execution process.
3. Explain the basic structure of a Java program.

Lecture – 5

Java Language Fundamentals: Tokens, Keywords, Identifiers, Comments, Data Types, Variables, Scope and Lifetime

Subject: Object-Oriented Programming with Java (DI03016051)

Unit: 1 – Introduction to Object-Oriented Programming Paradigm and Java

Learning Outcomes

After completing this lecture, students will be able to:

- Explain the meaning of Java tokens.
 - Identify keywords, identifiers and literals.
 - Use comments in Java programs.
 - Explain Java data types.
 - Declare and initialize variables.
 - Understand the scope and lifetime of variables.
-

1. Java Language Fundamentals

Before writing complete Java programs, we need to understand the basic building blocks of the Java language.

These building blocks include:

- Tokens
 - Keywords
 - Identifiers
 - Comments
 - Data Types
 - Variables
-

2. Java Tokens

A **token** is the smallest individual unit of a Java program that has a meaningful purpose.

Example

```
int marks = 85;
```

This statement contains tokens such as:

```
int  marks  =  85  ;
```

Main Types of Java Tokens

1. Keywords
 2. Identifiers
 3. Literals
 4. Operators
 5. Separators
-

3. Keywords

Keywords are **reserved words** that have a special meaning in Java. They cannot be used as names for variables, classes, or methods.

Examples

int
class
public
static
void
if
else
for
while
new
return

Example

```
int marks = 80;
```

Here, int is a keyword.

 **Remember:** Keywords have predefined meanings in Java.

4. Identifiers

An **identifier** is the name given to a program element such as:

- Class
- Variable
- Method
- Object

Examples

```
class Student {  
    int marks;  
}
```

Here:

- Student → Identifier
 - marks → Identifier
-

Rules for Identifiers

1. Can contain letters, digits, _ and \$.
2. Cannot start with a digit.
3. Cannot contain spaces.
4. Cannot be a Java keyword.
5. Java identifiers are case-sensitive.

Valid Identifiers

marks

studentName

_rollNo

totalMarks

Invalid Identifiers

2marks // Cannot start with a digit

student name // Space not allowed

class // Keyword

5. Comments

Comments are notes written inside a program for understanding the code.

The compiler ignores comments.

Single-Line Comment

// This is a single-line comment

Multi-Line Comment

/*

This is a

multi-line comment

*/

Documentation Comment

/**

* Documentation comment

*/

📌 Purpose of Comments:

- Explain the code.
 - Improve readability.
 - Help in program maintenance.
-

6. Data Types

A data type tells Java:

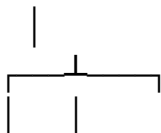
What type of data a variable can store.

For example:

- 25 → Integer
 - 3.14 → Decimal value
 - 'A' → Character
 - true → Boolean value
-

Types of Data Types in Java

Data Types



Primitive Non-Primitive

6.1 Primitive Data Types

Java has **8 primitive data types**.

Data Type	Example	Purpose
byte	100	Small integer
short	2000	Integer
int	50000	Common integer
long	100000L	Large integer
float	3.14f	Decimal value
double	3.14159	More precise decimal
char	'A'	Single character
boolean	true	True/False

Example

```
int age = 20;
double salary = 25000.50;
char grade = 'A';
boolean passed = true;
```

6.2 Non-Primitive Data Types

Examples:

- String
- Arrays
- Classes
- Interfaces

Example:

```
String name = "Rahul";
```

7. Variables

A **variable** is a named memory location used to store data.

Example

```
int marks = 85;
```

Here:

- int → Data type
- marks → Variable name
- 85 → Value

General Syntax

```
dataType variableName = value;
```

Example:

```
int age = 18;
```

Variable Declaration and Initialization

Declaration

```
int marks;
```

Memory is reserved for an integer variable.

Initialization

```
marks = 85;
```

A value is assigned to the variable.

Both Together

```
int marks = 85;
```

8. Types of Variables in Java

Variables can be classified according to where they are declared.

1. Local Variable

Declared inside a method or block.

```
void display() {  
    int marks = 80;  
}
```

2. Instance Variable

Declared inside a class but outside methods.

```
class Student {  
    int rollNo;  
}
```

Each object gets its own copy.

3. Static Variable

Declared using the static keyword.

```
class Student {  
    static String college = "ABC College";  
}
```

A static variable is commonly shared by all objects of a class.

 Detailed use of static will be studied later.

9. Scope of a Variable

Scope means the part of the program where a variable can be accessed.

Example

```
class Demo {  
    int x = 10;
```

```

void display() {
    int y = 20;
}
}

```

- x can be accessed by appropriate methods of the class.
- y can be accessed only inside the display() method.

Scope Examples

Variable Type Scope

Local Variable Within the method or block

Instance Variable Within the class through an object

Static Variable Shared within the class

10. Lifetime of a Variable

Lifetime means how long a variable remains in memory during program execution.

Variable Lifetime

Local Variable Until method/block execution ends

Instance Variable As long as its object exists

Static Variable Generally until the class/program remains loaded

Scope vs Lifetime

Scope

Where a variable can be accessed

Related to visibility

Lifetime

How long a variable exists

Related to memory existence

✚ **Important:** Scope and lifetime are related but not the same.

Example Program

```

class VariableDemo {

    int instanceValue = 10;
    static int staticValue = 20;

    void display() {

        int localValue = 30;

        System.out.println(instanceValue);
    }
}

```

```
    System.out.println(staticValue);
    System.out.println(localValue);
}
}
```

Here:

- instanceValue → Instance variable
- staticValue → Static variable
- localValue → Local variable

Key Points to Remember

- Tokens are the smallest meaningful elements of a Java program.
- Keywords have predefined meanings.
- Identifiers are names given to program elements.
- Comments are ignored by the compiler.
- Java has 8 primitive data types.
- A variable stores data in memory.
- Scope defines where a variable can be accessed.
- Lifetime defines how long a variable exists in memory.

Summary

Java programs are made up of tokens such as keywords, identifiers, literals, operators, and separators.

Data types specify the kind of data a variable can store. Variables provide names for memory locations.

Based on their declaration, Java variables can be:

- Local
- Instance
- Static

The **scope** of a variable defines where it can be used, while its **lifetime** defines how long it exists in memory.

GTU Examination Questions

Short Questions

1. What is a token?
2. Define a keyword.
3. What is an identifier?
4. What are comments?
5. List the primitive data types in Java.
6. Define a variable.
7. What is the scope of a variable?
8. What is the lifetime of a variable?

Long Questions

1. Explain the different types of Java tokens.
 2. Explain primitive data types in Java.
 3. Explain different types of variables in Java.
 4. Differentiate between scope and lifetime of a variable.
-

○

Lecture – 6

Operators, Type Conversion and Type Casting in Java

Subject: Object-Oriented Programming with Java (DI03016051)

Unit: 1 – Introduction to Object-Oriented Programming Paradigm and Java

Learning Outcomes

After completing this lecture, students will be able to:

- Define operators in Java.
 - Identify different types of operators.
 - Use arithmetic, relational, logical and other common operators.
 - Understand type conversion.
 - Differentiate between implicit conversion and explicit type casting.
-

1. What is an Operator?

An **operator** is a symbol that performs an operation on one or more values.

Example

```
int c = a + b;
```

Here:

- a and b → Operands
 - + → Operator
-

2. Types of Operators in Java

The commonly used operators are:

1. Arithmetic Operators
 2. Relational Operators
 3. Logical Operators
 4. Assignment Operators
 5. Unary Operators
 6. Conditional/Ternary Operator
 7. Bitwise Operators
-

3. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b

Operator	Meaning	Example
----------	---------	---------

%	Remainder	a % b
---	-----------	-------

Example

```
int a = 10;
```

```
int b = 3;
```

```
System.out.println(a + b); // 13
```

```
System.out.println(a - b); // 7
```

```
System.out.println(a * b); // 30
```

```
System.out.println(a / b); // 3
```

```
System.out.println(a % b); // 1
```

✦ **Important:** Integer division gives an integer result.

```
10 / 3 = 3
```

4. Relational Operators

Relational operators compare two values.

The result is always:

true or false

Operator Meaning

==	Equal to
----	----------

!=	Not equal to
----	--------------

>	Greater than
---	--------------

<	Less than
---	-----------

>=	Greater than or equal to
----	--------------------------

<=	Less than or equal to
----	-----------------------

Example

```
int a = 10;
```

```
int b = 20;
```

```
System.out.println(a < b); // true
```

```
System.out.println(a == b); // false
```

5. Logical Operators

Logical operators are used to combine conditions.

Operator Meaning

&&	Logical AND
----	-------------

,

!	Logical NOT
---	-------------

AND (&&)

Both conditions must be true.

age >= 18 && citizen == true

OR (||)

At least one condition must be true.

marks >= 40 || graceMarks == true

NOT (!)

Reverses the result.

!true = false

6. Assignment Operators

Assignment operators are used to assign values to variables.

Operator Example Equivalent To

=	a = 5	a = 5
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5

7. Unary Operators

Unary operators operate on only one operand.

Increment Operator

a++;

Increases the value by 1.

Decrement Operator

a--;

Decreases the value by 1.

Example

int a = 5;

a++;

System.out.println(a); // 6

a--;

System.out.println(a); // 5

Pre-Increment and Post-Increment

Pre-Increment

`++a;`

First increases the value, then uses it.

Post-Increment

`a++;`

First uses the value, then increases it.

Example:

`int a = 5;`

`int b = ++a;`

Result:

`a = 6, b = 6`

`int x = 5;`

`int y = x++;`

Result:

`x = 6, y = 5`

8. Conditional / Ternary Operator

The ternary operator is a short form of if-else.

Syntax

`condition ? value1 : value2;`

Example

`int age = 20;`

`String result = (age >= 18) ? "Eligible" : "Not Eligible";`

If the condition is true, "Eligible" is selected; otherwise, "Not Eligible" is selected.

9. Bitwise Operators

Bitwise operators work on the individual bits of integer values.

Common operators:

`&` AND

`|` OR

`^` XOR

`~` NOT

These are commonly used in:

- Low-level programming
- Device programming
- Performance-oriented applications

✦ For basic Java programming, only a basic understanding is required at this stage.

10. Type Conversion

Sometimes it is necessary to convert a value from one data type to another.

For example:

int → double

or:

double → int

This process is called **Type Conversion**.

There are two main types:

1. Implicit Type Conversion
2. Explicit Type Casting

11. Implicit Type Conversion

Implicit conversion happens automatically when a smaller data type is converted into a larger compatible data type.

This is also called **Widening Conversion**.

Example

```
int a = 10;
```

```
double b = a;
```

Here:

int → double

No special syntax is required.

General Order

byte → short → int → long → float → double

Example

```
int x = 25;
```

```
double y = x;
```

```
System.out.println(y); // 25.0
```

✦ No data is generally lost in widening conversion.

12. Explicit Type Casting

Explicit casting is used when a larger data type is converted into a smaller data type.

This is also called **Narrowing Conversion**.

Syntax

```
smallType variable = (smallType) value;
```

Example

```
double x = 10.75;
```

```
int y = (int) x;
```

Result:

```
y = 10
```

The decimal part is removed.

✦ **Important:** Narrowing conversion may result in data loss.

13. Widening vs Narrowing Conversion

Widening Conversion	Narrowing Conversion
Smaller → Larger data type	Larger → Smaller data type
Automatic	Requires explicit casting
Usually no data loss	May cause data loss
int → double	double → int

Example Program

```
class ConversionDemo {  
    public static void main(String[] args) {  
  
        int a = 10;  
        double b = a;    // Widening  
  
        double x = 10.75;  
        int y = (int) x;  // Narrowing  
  
        System.out.println(b);  
        System.out.println(y);  
    }  
}
```

Output

```
10.0  
10
```

Key Points to Remember

- Operators perform operations on operands.
 - Arithmetic operators perform calculations.
 - Relational operators return true or false.
 - Logical operators combine conditions.
 - Assignment operators assign or update values.
 - ++ and -- are increment and decrement operators.
 - The ternary operator is a short form of if-else.
 - Implicit conversion is automatic.
 - Explicit casting is done manually using (dataType).
 - Narrowing conversion may result in data loss.
-

Summary

Java provides various operators for calculations, comparisons, logical operations, assignments, and bit manipulation.

Type conversion allows values to be changed from one data type to another.

Widening:

Small Type → Large Type

Automatic

Narrowing:

Large Type → Small Type

Explicit Casting Required

GTU Examination Questions

Short Questions

1. What is an operator?
2. List different types of operators in Java.
3. What are relational operators?
4. What is the use of the ternary operator?
5. Define type conversion.
6. What is type casting?
7. Differentiate between widening and narrowing conversion.

Long Questions

1. Explain different types of operators in Java with examples.
 2. Explain implicit type conversion and explicit type casting.
 3. Differentiate between widening and narrowing conversion with examples.
-

Lecture – 7

Control Statements in Java

Subject: Object-Oriented Programming with Java (DI03016051)

Unit: 1 – Introduction to Object-Oriented Programming Paradigm and Java

Learning Outcomes

After completing this lecture, students will be able to:

- Explain the need for control statements.
 - Use conditional statements in Java.
 - Use if, if-else, nested if, and switch.
 - Use looping statements such as for, while, and do-while.
 - Understand the basic flow of execution in Java programs.
-

1. What are Control Statements?

Normally, a Java program executes statements **from top to bottom**.

However, real programs often need to:

- Make decisions.
- Repeat a task.
- Execute a particular block only when a condition is satisfied.

Control statements change the normal flow of program execution.

Example

Suppose a program needs to check whether a student has passed:

If marks ≥ 40

↓

PASS

Otherwise

↓

FAIL

2. Types of Control Statements

Control Statements



Conditional Looping Jump

In this lecture, we focus on:

Conditional Statements

- if
- if-else
- Nested if

- switch

Looping Statements

- for
 - while
 - do-while
-

3. if Statement

The if statement executes a block of code only when a condition is true.

Syntax

```
if (condition) {  
    // statements  
}
```

Example

```
int marks = 60;
```

```
if (marks >= 40) {  
    System.out.println("Pass");  
}
```

Output

Pass

If the condition is false, the block is skipped.

4. if-else Statement

The if-else statement provides two alternatives.

Syntax

```
if (condition) {  
    // if condition is true  
} else {  
    // if condition is false  
}
```

Example

```
int marks = 35;
```

```
if (marks >= 40) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}
```

Output

Fail

5. Nested if Statement

An if statement inside another if statement is called a **nested if**.

Example

```
int marks = 75;

if (marks >= 40) {

    if (marks >= 70) {
        System.out.println("Distinction");
    }

}
```

Output

Distinction

Real-Life Example

If student has passed

↓

If marks >= 70

↓

Distinction

6. switch Statement

The switch statement is used when one value needs to be compared with multiple possible values.

Syntax

```
switch (expression) {

    case value1:
        // statements
        break;

    case value2:
        // statements
        break;

    default:
        // statements
}
```

Example

```
int day = 2;
```

```
switch (day) {  
  
    case 1:  
        System.out.println("Monday");  
        break;  
  
    case 2:  
        System.out.println("Tuesday");  
        break;  
  
    default:  
        System.out.println("Invalid Day");  
}
```

Output

Tuesday

Role of break

The break statement terminates the switch block.

Without break, execution may continue into the next case.

Role of default

The default block executes when no case matches.

7. Looping Statements

A loop is used to execute a block of code repeatedly.

Example

Instead of writing:

Print Hello

Print Hello

Print Hello

Print Hello

Print Hello

we can use a loop.

8. for Loop

The for loop is commonly used when the number of repetitions is known.

Syntax

```
for (initialization; condition; update) {  
    // statements  
}
```

Example

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Output

```
1  
2  
3  
4  
5
```

Execution

Initialization

↓

Condition

↓

Body

↓

Update

↓

Condition

9. while Loop

The while loop executes as long as the condition is true.

Syntax

```
while (condition) {  
    // statements  
}
```

Example

```
int i = 1;
```

```
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}
```

10. do-while Loop

The do-while loop executes the block **at least once**, even if the condition is false.

Syntax

```
do {  
    // statements  
} while (condition);
```

Example

```
int i = 1;

do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

Important Difference: while vs do-while

while Loop	do-while Loop
Condition checked first	Body executes first
May execute zero times	Executes at least once
Entry-controlled loop	Exit-controlled loop

Example

```
int i = 10;

while (i < 5) {
    System.out.println("Hello");
}
```

Output:

No output

But:

```
int i = 10;

do {
    System.out.println("Hello");
} while (i < 5);
```

Output:

Hello

Comparison of Loops

Loop	Best Used When
for	Number of repetitions is known
while	Condition-based repetition
do-while	Code must execute at least once

Example: Simple Menu Program

```
int choice = 2;
```

```
switch (choice) {
```

case 1:

```
System.out.println("Add");  
break;
```

case 2:

```
System.out.println("Subtract");  
break;
```

default:

```
System.out.println("Invalid Choice");
```

```
}
```

This type of logic is commonly used in:

- ATM menus
- Billing systems
- Banking applications
- College management systems

Key Points to Remember

- Control statements change the normal flow of execution.
- if executes code when a condition is true.
- if-else provides two choices.
- Nested if contains one if inside another.
- switch is useful for multiple fixed choices.
- for is commonly used when the number of repetitions is known.
- while checks the condition before execution.
- do-while executes the body at least once.

Summary

Control statements are essential for decision-making and repetition in Java programs.

Conditional Statements

if

if-else

nested if

switch

Looping Statements

for

while

do-while

These statements allow Java programs to make decisions and perform repetitive tasks efficiently.

GTU Examination Questions

Short Questions

1. What is a control statement?
2. Explain the if statement.
3. What is a nested if statement?
4. What is the use of switch?
5. What is a loop?
6. Differentiate between while and do-while.

Long Questions

1. Explain different conditional statements in Java with examples.
 2. Explain for, while, and do-while loops.
 3. Differentiate between while and do-while loops.
-

Lecture – 8

Command Line Arguments and Java Memory Management

Subject: Object-Oriented Programming with Java (DI03016051)

Unit: 1 – Introduction to Object-Oriented Programming Paradigm and Java

Learning Outcomes

After completing this lecture, students will be able to:

- Explain command line arguments in Java.
 - Understand the role of String[] args in the main() method.
 - Explain Java memory areas such as Stack and Heap.
 - Understand the basic concept of Garbage Collection.
 - Review the important concepts covered in Unit 1.
-

1. Command Line Arguments

Command line arguments are values passed to a Java program when the program is executed.

These values are received through:

```
public static void main(String[] args)
```

Here, args is an array of strings that stores the values provided from the command line.

Example

Java Program

```
class Demo {  
    public static void main(String[] args) {  
        System.out.println(args[0]);  
    }  
}
```

Execution

```
java Demo Hello
```

Output

```
Hello
```

```
Here:
```

```
args[0] = "Hello"
```

2. Multiple Command Line Arguments

Program

```
class Demo {  
    public static void main(String[] args) {  
  
        System.out.println(args[0]);  
    }  
}
```

```
        System.out.println(args[1]);
    }
}
```

Execution

java Demo Java Programming

Output

Java

Programming

Array Indexing

args[0] → Java

args[1] → Programming

📌 **Important:** Array indexing starts from 0.

3. Command Line Arguments are Strings

All command line arguments are initially received as String values.

For example:

java Demo 10 20

The values 10 and 20 are received as strings.

To perform mathematical calculations, they must be converted into integers.

Example

```
class Addition {
    public static void main(String[] args) {

        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);

        System.out.println(a + b);
    }
}
```

Execution

java Addition 10 20

Output

30

4. What is Java Memory Management?

When a Java program runs, memory is required to store:

- Variables
- Objects
- Methods
- Program instructions

Java automatically manages memory using the **JVM**.

The programmer does not normally need to manually allocate and release memory.

5. Important Java Memory Areas

For basic understanding, the important memory areas are:

- Stack
 - Heap
-

6. Stack Memory

The **Stack** stores information related to method execution.

It commonly stores:

- Local variables
- Method calls
- References to objects

Example

```
void display() {  
    int x = 10;  
}
```

The local variable x is associated with the method's stack memory.

Simple Representation

Stack

main()

↓

display()

↓

Local Variables

When a method finishes execution, its stack memory is automatically released.

7. Heap Memory

The **Heap** is the area where objects are created and stored.

Example

```
Student s = new Student();
```

Here:

- s → Reference variable
- new Student() → Object created in Heap memory

Simple Representation

Stack Heap

s -----> Student Object

8. Stack vs Heap

Stack

Stores method-related data

Stores local variables

Memory is associated with method execution

Automatically managed

Heap

Stores objects

Stores instance data

Objects remain while they are reachable

Managed by Garbage Collector

9. Garbage Collection

In Java, objects that are no longer required can be removed from memory automatically. This process is called **Garbage Collection**.

Example

```
Student s = new Student();
```

```
s = null;
```

After:

```
s = null;
```

the previously created Student object may no longer have any reference.

It becomes eligible for garbage collection.

Reference → Object

↓

Reference removed

↓

Object becomes unreachable

↓

Garbage Collection

10. Why is Garbage Collection Useful?

Garbage Collection:

- Automatically removes unused objects.
- Frees memory.
- Reduces memory-related problems.
- Reduces the need for manual memory management.

✦ **Important:** The programmer can make an object eligible for garbage collection, but the JVM decides when the garbage collector actually runs.

11. Simple Memory Example

```
class Demo {
```

```
    public static void main(String[] args) {
```

```
        int x = 10;
```

```
        Student s = new Student();
    }
}
```

Conceptually:

Stack	Heap
-----	-----
x = 10	Student Object
s -----↑	

- x is a local variable.
- s is a reference variable.
- The Student object is created in Heap memory.

Unit 1 Quick Revision

1. OOP Concepts

The four main pillars of OOP are:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

2. Java

Java is:

- Object-oriented.
- Platform-independent.
- Portable.
- Secure.
- Robust.

3. Java Environment

JDK

↓

JRE

↓

JVM

- **JDK:** Used to develop Java programs.
- **JRE:** Used to run Java programs.
- **JVM:** Executes bytecode.

4. Java Program Execution

.java

↓
javac
↓
.class
↓
JVM
↓
Output

5. Basic Language Elements

Important concepts include:

- Keywords
 - Identifiers
 - Comments
 - Data Types
 - Variables
 - Operators
-

6. Type Conversion

Widening

Small Type → Large Type

Usually automatic.

Narrowing

Large Type → Small Type

Requires explicit casting.

7. Control Statements

Conditional

- if
- if-else
- Nested if
- switch

Looping

- for
 - while
 - do-while
-

Key Points to Remember

- Command line arguments are passed through `String[] args`.
- Command line arguments are initially stored as strings.
- `Integer.parseInt()` can convert a string into an integer.

- Stack memory is mainly associated with method execution and local variables.
 - Heap memory is used for object creation.
 - Garbage Collection automatically removes unused objects.
 - Java provides automatic memory management through the JVM.
-

Summary

Command line arguments allow values to be passed to a Java program during execution.

Java memory is automatically managed by the JVM. The **Stack** stores method-related information, while the **Heap** stores objects. When objects are no longer needed, they become eligible for **Garbage Collection**.

This lecture completes the major topics of **Unit 1**.

GTU Examination Questions

Short Questions

1. What are command line arguments?
2. What is the purpose of String[] args?
3. How are command line arguments converted into integers?
4. What is Stack memory?
5. What is Heap memory?
6. What is Garbage Collection?
7. When does an object become eligible for garbage collection?

Long Questions

1. Explain command line arguments in Java with an example.
2. Explain Stack and Heap memory in Java.
3. Explain Garbage Collection in Java.
4. Explain the important concepts covered in Unit 1.